

Every component of the system: what it is, what it does, and why it is that way

OCloud Solutions · state of the proof of concept on 11 September 2026 · written from the code as it runs, not from the plan

Custodian is an agent that runs a person's inbox for them. It reads what arrives, decides what needs the person, handles the rest without asking, and never sends anything as them without an exact typed yes. This document walks every part of it and, for each part, answers the same six questions: what is it, what does it do, how does it do it, why is it there, why is it built that way, and why not some other way.

2 doors into a mailbox — Gmail API, IMAP	17 pipeline stages, models only at four of them	4 models — three ours, one rented	9 database tables, none holding whole mail	16 ways a person meets it (scripts + one web page)
--	--	---	---	---

The one idea underneath everything

The more damage an action could cause, the more permission it needs. And the cheapest thing that can decide correctly is the thing that decides.

Those two sentences explain most of the choices below. Rules run before models because rules are free, instant and cannot be argued with. Models are used only where reading meaning is genuinely required. Anything irreversible — sending, forwarding, unsubscribing, trashing — stops at a gate that is written in our own code, below the models, where nothing inside an email can reach it.

How this document is organised

- A **The shape** — how the parts fit, in one picture
- B **The doors** — connectors: the interface, Gmail API, IMAP, and the factory that picks one
- C **The pipeline** — every stage, 1 to 19
- D **The memory** — the database, table by table
- E **The models** — four of them, with their honest state
- F **The surfaces** — how a person actually uses it
- G **The proofs** — the test scripts and what they establish

- H **The rules that cut across everything**
- I **What is not built, and what is still an open decision**

A · The shape

Five layers. Mail enters through a door, walks a fixed pipeline, leaves traces in a small database, and a person meets the result through one of several surfaces. Models sit at the leaves of the pipeline, never at its trunk.

A MAILBOX (Gmail, or anything that speaks IMAP)

↓

B · THE DOORS

connect.py picks one → connectors/gmail.py or connectors/imap.py |
Both answer the same questions. Neither is named above this line. |

| Envelope (sender, subject, date, what the sender declared)

↓

C · THE PIPELINE

3 sensitive? → 4 header facts → 2 strip → 5 forms → 7 dates |
→ 16 commitment → 8 READ (model) → 6 route → 9 needs a reply? |
→ 10 redact → 19 voice → 11 WRITE (model) → 12 restore |
→ 13 APPROVE (typed yes) → 14 act (15 unsubscribe, 17 quick) |

| decisions, actions, reminders, corrections

↓

D · THE MEMORY

store.py → mailbox.db: headers of everything, text of 30 days

↓

F · THE SURFACES

web.py (browser) chat.py (sentences) agent.py / watch.py |
brief · waiting · inbox · mailbox · correct · history · my_data |

E · THE MODELS sit inside stages 8, 9, 10, 11 only:

our classifier (8) · our reply gate (9) · our name finder (10) · Gemini (11, and 8 when

Why five layers and not two

The obvious build is one script that logs into Gmail, sends everything to a large model, and does what the model says. That was rejected for three reasons that each appear again below: it costs money on every email, it sends every email to an outside company, and in published testing every one of 1,404 email agents built that way was hijacked by instructions hidden in email text. The layers exist so that cost, privacy and safety are properties of the structure, not promises made in a prompt.

The order the stages actually run in

The stage numbers are the order of the original design. The order they *execute* in is different, on purpose. Stage 3 (is this sensitive?) runs before stage 2 (open and strip the body), because stage 2 opens the message and stage 3 exists to decide whether it may be opened at all. Stage 16 (commitments) runs straight after stage 7 (dates) and before stage 9 (needs a reply?), because a meeting reminder is needed whether or not the email

needs answering. Stage 6 (the router) reports after stage 8 because it is stage 8 that decides whether to escalate. Each of these is explained at its stage.

B • The doors

A connector is a thin adapter between one kind of mailbox and the rest of the system. There are two today. Nothing above them knows which one is in use.

B1 • The connector interface — `connectors/base.py`

WHAT IT IS

A contract: the list of questions every mailbox adapter must be able to answer, and a data shape called an **Envelope** that every adapter must produce.

WHAT IT DOES

Defines `account_email`, `list_ids`, `fetch_envelopes`, `fetch_raw`, `apply_label`, `archive`, `mark_read`, `trash`, and a set of **capability flags**: `supports_push`, `supports_labels`, `supports_categories`, `supports_threads`, `supports_send`, `supports_server_search`.

HOW

An Envelope carries only what can be known from headers: sender, subject, date, size, unread, which account it arrived at, whether the sender declared itself bulk or auto-submitted, the unsubscribe headers, and the threading headers. It has no body field, so an envelope-only scan cannot accidentally read a message.

WHY IT IS THERE

So the pipeline is written once. The rule is *ask a connector what it can do, never who it is*: code says `if conn.supports_push`, never `if conn.name == "gmail"`. A third kind of mailbox is then one new file.

WHY THIS WAY

The capability flags are not decoration. Some differences cannot be hidden — a mailbox that cannot push must be polled, and that is a different code path. Declaring the difference lets the caller choose the path honestly instead of failing later.

WHY NOT OTHERWISE

Hiding every difference behind one identical interface would force the weakest mailbox's behaviour on all of them. Exposing the provider's name would let provider-specific code leak upward, and the second provider would become a rewrite.

B2 • The Gmail API door — `connectors/gmail.py`

WHAT IT IS

The adapter for Google's official Gmail API, authenticated with OAuth.

WHAT IT DOES

Everything: lists, fetches headers in parallel (six worker threads, each with its own client because Google's client is not thread-safe), fetches bodies, applies and removes labels, archives, trashes, sends, drafts, forwards, rescues from spam, and asks Gmail *what changed since bookmark X* through `history.list` — one quota unit per poll instead of a full re-list.

HOW

Two OAuth scopes and deliberately not a third: `gmail.modify` (read, label, archive, trash — not permanent delete) and `gmail.send`. The full-access scope that permits `messages.delete` is never requested. *Not asking is stronger than promising not to.*

WHY IT IS THERE

It is the richest door: instant push is possible, labels and threads are native, and label removals the user makes in Gmail come back as events (the UC-29 correction feed).

WHY THIS WAY

Headers are requested by name (`From`, `To`, `Cc`, `Subject`, `Date`, `List-Unsubscribe`, `List-Unsubscribe-Post`, `List-Id`, `Precedence`, `Message-ID`, `In-Reply-To`, `References`, `Auto-Submitted`) — everything the pipeline needs, nothing more. Fewer bytes, less exposure.

THE COST

Google requires a paid annual security audit (CASA, roughly \$540–3,000 a year) before an app using the OAuth sign-in screen can serve more than 100 users, and tokens expire every seven days until then. That is a billing rule, not a technical limit, and it is why the second door exists.

B3 · The IMAP door — `connectors/imap.py`

WHAT IT IS

The adapter for IMAP, the protocol every mail server on earth speaks, authenticated with an **app password**. On Gmail it is much more than plain IMAP.

WHAT IT DOES

At connect time it asks the server `CAPABILITY`. If the answer contains `X-GM-EXT-1`, the server speaks Gmail's dialect, and three flags flip to true: real labels (`X-GM-LABELS`), real thread ids (`X-GM-THRID`), and the whole Gmail search language (`X-GM-RAW`) including `category:primary`. It reads headers in batches of 100 per command, reads bodies with `BODY.PEEK` so nothing is marked read, applies and removes labels, archives (a move to All Mail), trashes, rescues from spam with a single atomic `MOVE`, polls for new mail by highest UID, and sends over SMTP with the same app password.

HOW

Every command is a **UID** command. `imaplib`'s plain `fetch`, `search`, `store` and `copy` use sequence numbers — a message's position today, which shifts when anything above it moves. A stored position reused tomorrow names a different email, silently. UIDs are fixed for the life of the message.

WHY IT IS THERE

No audit, no fee, no approval wait, no 100-user cap, no expiring tokens. Measured on a real mailbox: 45 of 47 audited capabilities work, the Primary tab is reachable (1,748 messages), and a real message was sent in 4.9 seconds. The two genuine gaps are server push and snooze.

WHY THIS WAY

Capabilities are *detected*, never assumed from the hostname. Plug in a 1998 IMAP server and the Gmail flags stay false, the pipeline still runs, and only the extras are missing. Sending lives here too because the app password that opens `imap.gmail.com` also opens `smtp.gmail.com` — "IMAP cannot send" is true about the protocol and wrong about the product.

WHAT THE TESTS CHANGED

Seven defects in this file were found only by running it against a live server: label application that was silently archiving the message; sequence numbers stored as ids; a label parser that dropped any label without a space; a spam-folder lookup that never found Gmail's; missing Gmail folder names; folder creation with an unquoted name (every label with a space failed to exist); and archive that answered OK and did nothing. Each fix is documented in the file beside the code it changed, with the run that found it saved in `test_runs/`.

B4 • The factory — `connect.py`

WHAT IT IS

The one file that chooses a door.

WHAT IT DOES

`open_mailbox()` returns a Gmail connector or an IMAP connector. `--imap` forces IMAP, `--api` forces the API, and with neither it uses IMAP when an app password is set and no OAuth token exists. It also holds the `HOSTS` map (which IMAP server serves which domain) and three helpers every surface shares for reopening a stored message: `live_id`, `fetch_verified`, `looks_imap_uid`.

WHY IT IS THERE

So that no other file contains the words "gmail" or "imap". That is what makes "only the connection changes" true rather than claimed — and it was proved: the same email through both doors produced identical output, with only the connector line differing.

WHY THE HELPERS LIVE HERE

A stored `provider_id` belongs to the door that saved it. An API id means nothing to IMAP; an old IMAP id can name a different message. `fetch_verified` looks the message up by Message-ID when it has to and then checks the Message-ID *inside* what came back matches what was stored, refusing otherwise. One copy of that rule, used by the web page, the mailbox view, the brief and the backlog scanner, because five copies would be five chances to disagree.

C · The pipeline

One message at a time, through a fixed sequence. Each stage either attaches facts, exits early, or hands the message on. The rule for the whole pipeline: **a rule may route mail; it may never discard it.**

Stage 1 · Connect `rules`

What: the door, from Part B. **Does:** hands the pipeline an Envelope and says what it can do. **Why first:** the capability flags decide code paths downstream (poll or push; labels or folders). **Why not named:** the pipeline prints `push=`, `labels=`, never the provider.

Stage 3 · The sensitive gate `rules` `stage03_sensitive.py`

WHAT / DOES

Decides whether a message may be opened at all, seeing **only the sender and subject**. If it is sensitive, the pipeline stops here: the body is never fetched, not for a summary, not for a cache, not for anything. The person is told "a message from X arrived, not opened" and nothing more.

HOW

Four sources, not one list. Domain suffixes that cover whole categories at once (`.gov` `.gov.uk` `.gc.ca` `.gouv.qc.ca` `.mil` `.bank` `.insurance` `.nhs.uk`); a seed list of 27 bank, tax and payment domains; sender local-parts that are security channels (`otp`, `2fa`, `mfa`, `verification...`); subject patterns for password changes, sign-in alerts, verification codes and a six-digit code in the subject; security subdomains (`accounts.`, `security.`, `auth.`, `login.`, `id.`) whoever the provider is; and the user's own marks, read fresh from `user_sensitive.json` on every call.

WHY THIRD

Any later and something would already have opened the email before deciding it should not be opened. The ordering is the entire safety property, so the function signature enforces it: there is no parameter for a body.

WHY RULES, NOT A MODEL

We want certainty, not a probability, and a model would have to read the very thing we have decided not to read.

WHY NOT A SINGLE LIST

A finite list of banks will always miss one. Suffix rules and channel patterns cover categories; the user's marks cover their own bank; the list is only the seed. Real misses found in testing ("Your Instagram password has been changed", "New Steam Account Email Verification") were closed by widening the subject patterns to cover *confirmations* of a change, not only requests to make one.

Stage 4 · Header facts `rules` `stage04_headers.py`

WHAT / DOES

Reads what the sender declared about its own mail and turns it into **facts**: machine-generated (`Precedence: bulk`), has an unsubscribe header, one-click unsubscribe (RFC 8058), is a reply, is an

auto-reply (`Auto-Submitted`), and whether our provider's authentication stamp passed.

THE ONE EXIT

`engagement_exit` : if this person has received at least eight messages from a sender and opened none, the message leaves the pipeline. That is a fact about the person's behaviour, not a guess about the sender.

WHY FACTS AND NOT VERDICTS

"Machine-generated" is a fact. "Junk" is not — bank statements, flight bookings, invoices and password resets are all machine-generated, and a shop's sale email is noise to one person and the point of the mailbox to another. This stage attaches facts; the decision belongs to stage 9.

WHY THE AUTHENTICATION CHECK IS CAREFUL

Only the topmost `Authentication-Results` header, and only if *our own* provider stamped it, is trusted. Every header below it was written by whoever sent the mail. Parsing that header without checking who stamped it is an exploitable bug, not a style question.

WHAT TESTING ADDED

Three headers, not one. Checking only `Precedence: bulk` let bounces (which set `Auto-Submitted`) and marketing platforms (which set `List-Unsubscribe` but not `bulk`) through as if a person had written them. A person's mail client has never once added a `List-Unsubscribe` header.

Stage 2 · Parse and strip `rules` `stage02_strip.py`

WHAT / DOES

Now, and only now, the body is fetched. MIME is taken apart, HTML is turned into text, and the quoted previous conversation and the signature are cut away, keeping only the new words.

HOW

`selectolax` for HTML (a C-backed parser, far faster than BeautifulSoup on marketing mail) and `email-reply-parser` for the quoted history.

WHY

It is the single biggest cost saving in the system. Published measurement on 938 real emails put the reduction at about 85%, median 966 tokens down to 124. Measured here: one real message went from 156,130 raw bytes to 5,399 characters. Every model downstream reads the short version.

WHY AFTER 3 AND 4

Because it opens the message, and stages 3 and 4 decide whether to.

Stage 5 · Forms and layouts `rules` `stage05_forms.py`

WHAT / DOES

Two mechanisms sharing a stage. **Half one:** the answer arrived attached — a calendar invite (`.ics`) carries the exact time, place and attendees; an Apple Wallet pass (`.pkpass` , a ZIP with JSON inside) carries flight, gate and seat; some invoice PDFs carry the whole invoice as embedded XML (Factur-X / ZUGFeRD). We open the file and read the fields; nothing is interpreted. **Half two:** a layout fingerprint — your electricity bill looks identical every month, so after a few of them the shape is recorded and no model is ever needed for that sender again.

WHY NO AI

These are data, not prose. A model reading a calendar invite can only be as right as the file, and can be wrong. Reading the file directly cannot be wrong.

WHY IT MATTERS DOWNSTREAM

UC-23: a category label (flight, invoice, meeting) is applied only when the sender *declared* it through one of these files, never from text the model guessed at. A flight label on a message we merely guessed was about a flight is worse than no label — the Owner stops trusting the folder.

Stage 7 • Find dates rules `stage07_dates.py`

WHAT / DOES

Finds phrases in the text that refer to a time, resolves them to a real date, and notes whether a deadline cue (*by, before, due, no later than...*) sits near each.

HOW

Three jobs, three tools: **find** the phrases with `dateparser` plus our own patterns for relative phrases it misses ("next week", "end of month"); **pick** which one is the real deadline with a model (stage 11's helper); **calculate** the date with Python, never a model.

WHY NEVER A MODEL FOR ARITHMETIC

On temporal arithmetic BERT-base scores 25.9 and RoBERTa-large 29.1 against a random baseline of 35.4 — worse than guessing. A model may read that a sentence refers to a time; code works out which time.

WHAT TESTING ADDED

`dateparser` is very loose. On real mail it returned "dil" and "bla" from an Urdu song lyric and "150, 41 Comments" from a LinkedIn post as dates, and set reminders for them. A candidate must now *look like* a date — contain a number, a month, a weekday or a relative word — and a bare number is not a date.

Stage 16 • Commitments rules `stage16_commitments.py`

WHAT / DOES

Turns a date into a reminder that fires. Classifies the kind (meeting, appointment, bill, application, renewal, deadline) and warns by type: two hours before a meeting, a day before an appointment, three days before a bill, six weeks before a passport renewal.

WHY IT RUNS HERE, OFF STAGE 7

"Your meeting is tomorrow at 3" needs no reply — stage 9 correctly says so — and it is exactly the message a person most needs surfaced. Reminders were once computed inside the drafting path, so they only happened for mail that needed answering. Reply and remember are different jobs.

WHY NO APPROVAL

A reminder is reversible; the worst case is a notification the user dismisses. The gate is spent where it is needed.

WHY AN ATTACHMENT WINS

An `.ics` file is the sender stating the time in a machine-readable field; a phrase in a sentence is us inferring it. When both exist the declared one wins — and it is the only source allowed for machine-sent

mail, because "80% off today" from a marketing sender once produced a reminder due at 13:11 that fired at 13:16.

WHY THE KEYWORD ORDER IS LOAD-BEARING

"Passport renewal" was classed as a bill (three-day warning) because bill's list contained "renew" and was checked first. Specific kinds are checked before general ones.

LEARNING

BR-88: repeatedly dismissing a warning as too early moves it later; acting on it at once moves it earlier. Stored as a correction, not a model weight, so it takes effect on the next reminder.

Stage 8 • Read it `model` `local_classifier.py` • `model.py`

WHAT / DOES

The only place anything reads an email to understand it. It answers one question: does this need a response from the person? Our own model answers first; a rented model answers only when ours is unsure.

HOW

An XLM-RoBERTa-base classifier (278M parameters, multilingual) fine-tuned on this mailbox, loaded once and held in memory, about 110 ms per email on a laptop CPU, text never leaving the machine. Topic ("work", "money or bills", "account or security"...) is a coarse bucket, so rules do it — free and instant, no second model to keep in step.

WHY THIS AND NOT A LARGE MODEL FOR EVERYTHING

Gemini takes 1–2 seconds, costs money per email, and the text leaves. Ours is free and private. The two sit together through stage 6.

HONEST STATE

Trained on 199 examples from one mailbox. Strong on the classes with data, weak on the ones with two examples each. It proves the component works end to end; it is not yet good enough to ship. What improves it is more labelled data and the correction loop.

Stage 6 • The router `rules` `model.py`

WHAT / DOES

Decides whether to trust our model's answer or escalate to the rented one. The rule: escalate when our model's **confidence is below 0.65**. Never on subject matter, never on category.

WHY CONFIDENCE AND NOT CATEGORY

A category rule ("send all legal mail to the big model") would send the same mail every time regardless of whether the small model could handle it. Confidence sends only what the small model genuinely cannot do, which is the only spend that buys anything.

WHAT IT CAUGHT ON ITS FIRST RUN

Our model called a song lyric a request at 0.53; the router escalated; Gemini answered "no response needed" at 0.90.

DEGRADES, DOES NOT STOP

With no Gemini key it keeps our answer and says so. With Gemini failing it keeps our answer and says so.

Stage 9 • Needs a reply? `stage09_gate.py`

WHAT / DOES

Answers "will this person actually act on this?" **without re-reading the email**. It combines stage 8's meaning with history: have they replied to this sender before, opened them before, is it addressed only to them, is it in a thread, is the sender verified, is it business hours, is it a request.

HOW

Hand-set weights give a probability. If a trained model exists for this mailbox (LightGBM, 17 features, trained on what this person opened) its score is **averaged** with the hand-set one, not substituted. The verdict is "needs a reply" when the probability is at least 0.45 *and* stage 8 said it was a request. Every verdict carries its reasons, written at the moment it is made.

WHY HISTORY BEATS WORDS

Measured on real enterprise mail, feature groups on their own scored: history with the sender 0.692, what the email was understood to mean 0.595, timing 0.540, **the raw words 0.511** — chance is 0.500. Everything combined: 0.721. So stage 8 reads once; stage 9 decides using what it understood plus history.

WHY BLENDED, NOT REPLACED

The trained model's AUC is 0.684, below the published 0.72–0.79 ceiling, and it learned "opened", which is a slightly different question from "replied". Two mediocre estimates that disagree for different reasons beat either alone, and a bad training run can never swing a decision on its own.

THE BUG THIS STAGE TAUGHT

It compared stage 8's label against a vocabulary written for an older four-way classifier. When stage 8 became binary its label was not in the set, so "is a request" was always false, so nothing was ever "needs a reply" — while printing a plausible probability every time. Two components each doing what they were written to do, joined by a string comparison nobody re-checked.

Stage 10 • Redact `stage10_redact.py` • `local_pii.py`

WHAT / DOES

The only door out of the building. Names, email addresses, card numbers, IBANs, national identifiers (CNIC, NI, SSN, SIN), phone numbers and URLs are replaced with placeholders before any text reaches an outside model. The text is also cut to the least that will do the job (3,000 characters).

HOW

Patterns first, then the model. Structured identifiers have checkable structure, so a regular expression gets them essentially right every time; the model once called a credit card an SSN. Names have no structure, so only the model can find them. Patterns claim what they are certain about; the model takes what is left. The mapping from placeholder to real value is held in memory for the life of the request and never written to disk.

WHY ONE DOOR

With one exit we can show what left; with several we can only hope.

THE HONEST LIMIT

On a benchmark of real documents, a model self-reporting 98.82 F1 scored 0.27, and nothing tested beat a non-expert human at 0.77. So the claim we can defend is "we strip identifiers, we minimise what crosses, and we contract for zero retention" — not "names are removed".

Stage 19 • Your voice `rules` `stage19_voice.py` • `learn_voice.py`

WHAT / DOES

Learns how this person writes, from their **Sent** folder only. Two things are stored: a **profile** (how they open, how they sign off, how long they write, how formal) made by plain counting, and **examples** — their own past replies, each stored as a vector so the four or five closest to the current situation can be handed to the writing model as samples.

WHY NOT A MODEL TRAINED ON THE USER

Nothing about them is baked into any model's weights. Deleting the account removes it completely — there is no model to unlearn.

WHY NO SEPARATE VECTOR DATABASE

Thousands of vectors per user and a few searches a day is not a job for a second system. POC: TF-IDF vectors as blobs in SQLite, compared in numpy. Production: the same vectors in Postgres with pgvector. "No separate vector database" is not "no vectors".

Stage 11 • Write it `model` `model.py` • `prompts.md`

WHAT / DOES

Drafts a reply from the redacted text, the profile and the examples. It never sends. It is the only paid step in the pipeline.

HOW

Gemini (`gemini-flash-latest` , with three fallbacks when the free tier is busy), using the `draft_reply` prompt from `prompts.md` . Every prompt the system ever gives a model is in that one file, editable without a developer, because wording is a product decision.

WHY RENTED HERE AND NOT OWNED

Reading is a classification, which a small model does well. Writing in a person's voice is generation, which small models do badly. Production decides between Claude and Gemini by blind comparison on real drafts, not by benchmarks.

Stage 12 • Names back `rules`

The placeholders are swapped back for the real values, on our side, before the person ever sees the draft. If any placeholder cannot be resolved the draft is discarded rather than shown half-restored.

Stage 13 • Approve `rules` `stage13_approval.py`

WHAT / DOES

The most important code in the system. Nothing irreversible — send, forward, unsubscribe, trash — happens without an exact typed yes. Reversible things — label, archive, mark read, snooze, remind, rescue from spam, *draft* — happen freely and are logged with how to undo them.

HOW

The draft is hashed. The yes is bound to that hash, so editing the draft voids the earlier yes. The yes must be an *exact* match against a short list ("yes", "send it", "haan", "ji", "theek hai"...), never a substring — "don't send it" contains "send it". Approval expires after 24 hours. The default, for anything ambiguous, is that nothing happens.

WHY IN CODE AND NOT IN A PROMPT

In published testing all 1,404 real email agents tested were hijacked, most within about two attempts, and the attack works by telling the model it need not ask. A model's manners are a behaviour, and behaviours can be argued with. This code never reads the email, so nothing inside an email can reach it.

WHY A DRAFT NEEDS NO APPROVAL

UC-25: a draft in the Owner's own Drafts folder cannot leave without them pressing send in their own mail app. The approval is structural rather than procedural, which is stronger.

Stage 14 • Act `rules` `stage13_approval.execute()`

One connector call per decision. The audit row is written *before* the call, with how to reverse it — if the process dies in between, we have a record of an action that may not have happened, which is the correct direction to fail. A send with no recipient is refused before anyone is asked to approve it, because asking someone to approve a send that cannot work trains them to click yes on things that do nothing. The send line itself is the only line in the project that puts mail into the world, and it is reachable only through the check above.

Stage 15 • Unsubscribe `rules` `stage15_unsubscribe.py`

Irreversible, and that is not obvious. You cannot un-unsubscribe. So it sits with send behind a typed yes. Two mechanisms exist and only one is safe: RFC 8058 one-click (a single HTTPS POST to a header the sender advertised) is done; a link in the body (a page with a button somewhere, possibly a login, a survey, a tracking beacon that fires on open) is refused and handed to the person instead. Refusing to act is a real answer.

Stage 17 • Quick replies `rules` `stage17_quick.py`

UC-26. A surprising share of replies are one word — "Yes", "Thanks", "Received". Sending those through a language model is paying for prose where none is wanted. A fixed list of eight answers, suggested by what the message asked, at most three at a time, no model, no cost. The tap is the approval (BR-96), and the exact text is shown before it goes.

Why there is no stage 18. The numbering is the design's; stages were added as use cases arrived (15, 16, 17, 19) and one number was never assigned. Renumbering would break every reference in the documents, the meeting notes and the code comments for no gain.

D • The memory

One SQLite file, `mailbox.db`, behind one module, `store.py`. The design in one sentence: **headers of everything, ever; the text of the last 30 days, important only; everything older fetched live when opened.**

TABLE	WHAT IT HOLDS	WHY IT IS SHAPED THAT WAY
messages	One row per envelope: sender, subject, date (raw and ISO), size, unread, in inbox, categories, the unsubscribe and threading headers, recipients, which account. Deliberately no body column.	About 1 KB a message; ten years fits in a few hundred MB. The missing column is the privacy promise as a fact about the schema, not a policy page. <code>date_iso</code> exists because sorting the raw RFC date as text sorted alphabetically — every "newest first" query in the project was wrong and looked plausible.
bodies	Message text for a bounded window: last 30 days, only messages worth reading (not sensitive, not machine-sent, not auto-replies, not from dormant senders). Currently 11 rows, 0.01 MB.	Retention is enforced in code — <code>trim_bodies()</code> runs every time the view opens. "We hold 30 days of your important mail" is a sentence a customer can weigh; "we hold all your mail" is not.
decisions	Every verdict, by stage, with the reason and score, written at the moment it was made.	BR-105. Reconstructing a reason afterwards is guessing wearing a record's clothes. A decision that is not written down did not happen — a watcher once printed verdicts for weeks and recorded none.
actions	Every action taken, with the state before it, and whether it has been undone.	BR-108/109. Undo is possible only because "before" was kept. An undo is itself an action and lands here too; history is never silently rewritten.
corrections	What the Owner told us we got wrong: scope (sender, domain, one message), what it was, what it should be, where the correction came from.	BR-106/107. Looked up <i>before</i> any model is trusted, so a correction changes behaviour on the very next email, not after retraining. Corrections are the only training data we can clearly use.
reminders	Reminders and snoozes: due time (computed in code, never by a model), kind, why.	Both are "show me this again at a time", so one table with a kind, not two that drift apart.
handled	What has already been done to a message, so a second run does not do it twice.	A message is a fact about the mailbox; an action is a fact about us. Different lifetimes, different tables.
style_examples style_profile	The person's own past replies as vectors, and the counted profile of how they write.	Stage 19. The only user text ever held, and it is only their own outgoing writing.

Why SQLite and not Postgres

One file, zero setup, in the standard library, and every query in the POC is a few thousand rows. Production is Postgres — the same schema, with the vector blobs becoming a pgvector column.

Nothing in the queries depends on SQLite; the swap is the connection string and a migration.

Why Message-ID is the key, and provider_id is a handle

The Message-ID header is written by the sender's mail system and is the only identifier that means the same email in Gmail, Outlook, JMAP and IMAP. The provider's own id is a handle that belongs to one door — and on IMAP it belongs to one folder. The same message CC'd to two of the Owner's accounts is one row when shown and two copies when acted on, matched on Message-ID (BR-99).

E • The models

Four. Three are ours, run on this laptop, cost nothing per email, and never let the text leave. One is rented. Each is used for exactly one job, and each job is the one that nothing cheaper could do.

MODEL	STAGE	WHAT	TRAINED ON	HONEST STATE
Our classifier classifier/	8	XLM-RoBERTa-base, 278M parameters, multilingual. Binary: <i>needs a response from you / no response needed</i> . ~110 ms per email on CPU.	199 emails from this mailbox, labelled by Gemini (distillation) and by hand, fine-tuned on Kaggle's free T4 GPU in minutes.	Works end to end; too little data to ship. mDeBERTa-v3 was tried first and collapsed to one class after one epoch (loss went to NaN); XLM-R is the same size and stable.
Our reply gate gate_model.txt	9	LightGBM, 300 trees, 17 features (sender history, machine markers, subject shape, time of day, first contact...). Under 1 MB. Trains in 0.24 s.	5,895 messages from this mailbox with answers the mailbox already knew (opened / replied). Nobody labelled anything.	AUC 0.684 against a published ceiling of 0.72–0.79. Blended 50/50 with hand-set weights. One model per person, retrained weekly in production.
Our name finder pii_model/	10	XLM-RoBERTa-base token classifier. 113 fine labels collapsed to a handful of families (PERSON, EMAIL, PHONE, ADDRESS, DATE, ORG...).	6,000 samples of the public ai4privacy/pii-masking-200k dataset. Nothing of ours involved.	0.922 on identifier tokens in its own test set. Runs after the regex patterns, never before. The 44M DeBERTa-v3-small in the plan scores within 0.001 of models ten times its size; the trained artefact here is XLM-R for stability.
Gemini gemini-flash-latest	11, and 8 when unsure	Rented. Writes the draft; reads an email only when our classifier's confidence is under 0.65. Sees redacted text only.	—	Free tier with three fallback models when busy. Production chooses between Claude and Gemini by blind comparison on real drafts.

Why the models are at the leaves

Every stage that can be rules is rules: sensitivity, header facts, stripping, forms, date arithmetic, reminders, the approval gate, unsubscribe, quick replies. The models are spent on exactly three things a rule cannot do — read what a person meant, weigh many weak signals about whether they will act, find a name in free text — and one thing a small model does badly, writing in someone's voice. That is why the pipeline is deterministic with models at the leaves, and why a wrong model answer is always caught or overridden by a rule (a machine-sent flash sale scored "needs a reply" at stage 9 and was still shown as machine mail, because stage 4's fact wins).

How training data was got without a labelling budget

The gate: the mailbox already knows the answers (opened, replied), so 30,000 messages are 30,000 examples the moment the scan finishes. **The classifier:** distillation — a large model labels ten emails per request (`batch_label.py`), a small one learns from those labels, and the small one inherits the large one's mistakes until real corrections improve it. Then careful hand labels (`dump_for_labelling.py` → `labels.txt` → `apply_labels.py`), because the classifier is shared across every user and is labelled once, ever. **The name finder:** a public dataset already marked up.

What is planned and not done: shrinking

`shrink_model.py` converts the classifier to ONNX at 8 bits: from 1.1 GB PyTorch to roughly 280 MB, measured elsewhere at about 14× faster on CPU, and production never installs PyTorch. Written; not yet run.

F • The surfaces

How a person meets the system. Every one of these reads the same database and calls the same pipeline; none contains a second copy of any decision.

SURFACE	WHAT IT IS FOR	WHY IT EXISTS AS ITS OWN THING
web.py	The mailbox in a browser at <code>localhost:8000</code> : every email with its verdict and reason, the door in use, the storage counts, click to open (from the 30-day store or fetched live in under a second), a "how it works" page with live numbers, and <code>--watch</code> to run the agent in the background while the page is open.	Sir: "we are definitely going to give the view of the mailbox to the user — first the basic thing, then the agentic experience on top." Built on Python's own <code>http.server</code> , no framework, so it runs with nothing installed. It only reads; the background agent is a separate thread with its own connection.
chat.py	Sentences, not commands: "email ali and ask if he got the report", "what's in my inbox", "sort my mail".	Sir mandated text input, not settings forms. What the person types is an <i>instruction</i> and what arrives in mail is <i>data</i> ; they are read by different code (<code>user_input.py</code>), and the model is only ever handed a multiple-choice question our code built from the current state — with no draft on screen, "send" is not an option.
agent.py	The full pipeline over the newest N messages, printing every stage and its reason.	The teaching surface. Every decision is shown with why, so it can be justified stage by stage.
watch.py	Live: polls for new mail every 15 s, runs the pipeline, labels, optionally drafts and asks.	Polling, not push, on a laptop: push needs a public HTTPS endpoint. <code>history.list</code> costs one unit per poll against ~1,500 for a re-list.
scan.py	Catches up on the backlog — messages that have never been through the pipeline.	The watcher starts from "now"; nothing else would ever scan the 7,000 behind it. It contains no pipeline: it calls <code>agent.run_one</code> , so backlog and live can never reach different verdicts.
send_mail.py	Send a mail on the person's behalf, through the real gate.	It has no shortcut to the send call; a second path to sending is a second place the gate can be missed.
brief.py	The daily brief: what needs you today, with at most eight one-line summaries; <code>--email</code> sends it to yourself (UC-31).	It deliberately does not summarise the whole mailbox. 7,859 paid calls to produce a wall nobody reads; the brief summarises only what survived the pipeline and caps the count so the bill cannot surprise anyone.
waiting.py	UC-22: what you asked for and never got back. Thresholds scale with how often you correspond (2 days for a frequent contact, 7 for a stranger).	The one failure that leaves no trace in the inbox. BR-83: the reminder goes to the Owner, never a chase to the recipient. <i>Gap: API door only today.</i>

inbox.py	UC-27/28: every account in one list; one search across all of them; duplicates collapsed on Message-ID; <code>--capabilities</code> shows what each door can do.	Search is over our index (sender, subject, date, category, account), not inside message text — BR-103 leaves full-text search an open decision and this file says so rather than quietly deciding it.
mailbox.py	The three-tier view in the terminal; <code>--warm</code> fills the 30-day text store.	The design Umar specified: headers for everything, bodies for 30 days, older fetched live (measured 0.7–0.8 s).
correct.py	UC-29: tell it it was wrong — always/never needs a reply, never open this, it is not sensitive, wrong label, stop labelling; scoped to a sender, a domain or one message; <code>--scan</code> picks up labels removed in Gmail; <code>--timing</code> fixes reminder lead times.	Takes effect on the next email. "Only this one message" once stored the sender's address and so behaved as "this sender forever" — now pinned to the Message-ID.
history.py	UC-30: everything the system has done, newest first, and <code>--undo N</code> .	Undo is cheap because we only ever move things. Send, forward and unsubscribe say plainly they cannot be reversed.
my_data.py	UC-32: export everything we hold, or erase it.	The mail itself was never held, so there is nothing to export there — a fact about the schema. Erasure changes nothing in the mailbox (BR-116), and the export names what survives (aggregate sender reputation) rather than hiding it (BR-117).
sort_mailbox.py	Applies labels in the mailbox; <code>--undo</code> removes exactly what it added.	The only script that changes the mailbox in bulk, and deliberately the smallest. Labels only; never moves, archives or deletes.
run.py / analyze.py / report.py	The first scan: every envelope into the database, then measured findings written to <code>report.html</code> .	No model runs; every finding is a query over headers. "Never once opened" is a fact about behaviour, not a guess about the sender.
learn_voice.py	Builds the writing profile and examples from the Sent folder.	Stage 19's data. The only place the user's own text is stored.

G • The proofs

Scripts that ask the real server and save what it said. They exist because a comparison document tells you what someone believes a protocol can do; running it found six defects in our own code that the terminal had made look correct.

SCRIPT	WHAT IT ESTABLISHES	RESULT ON THIS MAILBOX
<code>audit_imap.py</code>	47 capabilities over IMAP, across nine areas: what the server advertises, every label, the Gmail tabs, spam and special folders, reading one message without marking it read, batched fetch matching one-at-a-time, live polling, search, and every action that changes the mailbox — run on messages the audit writes itself and deletes afterwards.	45 work, 2 do not (push, snooze). Inbox counted before and after: unchanged.
<code>verify_doc.py</code>	The IMAP-vs-API comparison notes, claim by claim. Sends one real mail with <code>--send</code> .	6 confirmed, 3 wrong (labels, categories, Important marker are all reachable), 3 partly right, 4 policy. The duplicate trap shown on a real message in four folders, collapsed to one row by our code.
<code>limits_imap.py</code>	Google's published caps quoted, never hit; this mailbox weighed message by message; fetch speed one-at-a-time versus batched versus our code; the connection cap probed and every connection closed.	7,882 messages, 780 MB. Our design downloads 59 MB = 2.3% of one day's cap. Our fetch runs at ~100 messages/s, 28× the old speed. 16 connections held, none refused.
<code>test_imap.py</code>	The first proof: Gmail over an app password, the Primary tab over IMAP, the same pipeline through the other door.	Primary 1,747 at the time; identical pipeline output through both doors.
<code>runlog.py</code>	Every run of every proof saved verbatim to <code>test_runs/</code> , timestamped, one file per run.	The evidence is a folder of files with the time each happened, not a paragraph saying so.

H • The rules that cut across everything

Eight rules that no single component owns and every component obeys.

1	Instruction vs data	Text the user types is an instruction and goes to <code>user_input.py</code> . Text inside an email is data and goes to stages 2, 8, 10, 11. They never share a code path, which is why an email cannot issue a command.
2	The gate is code	Nothing irreversible without an exact typed yes bound to a hash of the exact text, checked by Python that never reads the email. Not a prompt, not a model's manners.
3	One door out	Everything that reaches an outside model passes through stage 10. Redaction mapping lives in memory only.
4	No body column	The messages table cannot hold mail. The bodies table is the bounded, auto-trimmed, 30-day exception, and it excludes sensitive mail by construction.
5	Capability, not identity	Ask a connector what it can do. No file outside <code>connect.py</code> and <code>connectors/</code> names a provider.
6	Message-ID is the key	The only identifier that means the same email on every door and every day. Provider ids are handles that belong to one door, and are verified on reopen.
7	Rules before models; models at the leaves	A rule that fires is free, instant and predictable. Models are spent only where nothing cheaper can decide. A rule's fact always wins over a model's guess.
8	Move, never delete	The system never permanently deletes and never asks for the permission that could. Every action is logged with its "before", so undo is by definition rather than by effort. A decision that is not written down did not happen.

And the working method that produced the code

Every non-obvious choice is written beside the code it explains, marked 🚩, in the words a reviewer would need: what was wrong, how it was found, why the fix is this and not the other. A test that talks to the real server is written before a claim is repeated. When a test finds that the system changed something it should not have, the run is kept, the change is reverted, and the test is made to check its own claim afterwards.

I · What is not built, and what is still an open decision

ITEM	STATE	NOTE
Outlook / Graph, JMAP, Zoho connectors	not started	Sir: Google only for now. Each is one new file behind the same interface.
Gmail push (Pub/Sub)	not started	Needs a public endpoint. Polling every 15–20 s covers the POC.
IMAP IDLE	not wired	Advertised by the server; holds one socket per mailbox. Deferred until the connection model for many users is decided.
waiting.py over IMAP	gap	Uses Gmail-API thread calls directly. Needs the Sent folder and threading over IMAP.
forward() over IMAP in the executor	built and tested	Sent in 4.7–8.6 s to the owner's own address during the audit.
ONNX / 8-bit classifier	written, not run	<code>shrink_model.py</code> .
AC-29.1 — decision reason on each mailbox row	on the web page	Every row shows the reason under the subject.
WhatsApp as the channel	not started	The design's chosen channel to reach the human. The chat surface is the same logic on a terminal.
More training data for the classifier	ongoing	199 examples proves the loop; a few hundred careful labels are needed before the model is trusted alone.
BR-103 — full-text search	open decision	Searching inside message text means storing every message's words, which contradicts "we never held it". Needs Umar and sir, not code. The 30-day store is a bounded partial answer.
The exposed Gemini key	confirm	A key was pasted into a chat earlier in the project. Its deletion and replacement has not been confirmed.